

Support Vector Machines from Scratch

Line-by-line theoretical commentary on `svm_from_scratch.py`

This document is a companion to `svm_from_scratch.py` in the *finance-course* repository. Each code block from the script is reproduced verbatim and followed by the theory behind it: the underlying math, the connection to Igor Halperin's *Supervised, Unsupervised and Reinforcement Learning in Finance* slides (NYU Tandon, 2017), and the practical pitfalls.

The slides are organised in three parts:

Part I — primal SVR / SVC, ϵ -insensitive loss, soft margin C .

Part II — KKT conditions, the dual problem, the support-vector expansion, convex optimisation.

Part III — the kernel trick: features $\Phi(x)$ replaced by a kernel $k(x, x')$.

0. Mathematical recap

Primal soft-margin SVC (Part I)

The primal problem minimises the squared norm of the weight vector (equivalently, maximises the margin width $2/\|w\|$) plus a penalty C times the total slack:

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s. t.} \quad & y_i(\langle w, x_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

Here $y_i \in \{-1, +1\}$, ξ_i is the per-sample slack, and $C > 0$ controls the trade-off between margin width and tolerance for margin violations.

Lagrangian

Introducing multipliers $\alpha_i \geq 0$ on the margin constraints and $\mu_i \geq 0$ on the slack-non-negativity constraints:

$$L(w, b, \xi, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(\langle w, x_i \rangle + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i$$

Stationarity conditions

Setting the partial derivatives of L to zero gives:

$$\partial L / \partial w = 0 \Rightarrow w = \sum_i \alpha_i y_i x_i$$

$$\partial L / \partial b = 0 \Rightarrow \sum_i \alpha_i y_i = 0$$

$$\partial L / \partial \xi_i = 0 \Rightarrow C - \alpha_i - \mu_i = 0 \Rightarrow 0 \leq \alpha_i \leq C$$

The dual problem (Part II)

Substituting the stationarity conditions back into L collapses the primal into a pure function of α , the **convex quadratic programme** at the heart of every SVM:

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(x_i, x_j) \\ \text{s. t.} \quad & \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C \end{aligned}$$

The kernel trick (Part III)

Because the dual depends on the data only through pairwise dot products, we can replace every $\langle x_i, x_j \rangle$ by a Mercer-admissible kernel $k(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$ for some implicit feature map Φ . The script uses the simplest possible kernel — the **linear** one:

$$k(x, x') = \langle x, x' \rangle = x^\top x'$$

which corresponds to $\Phi(x) = x$, the identity. Because Φ is finite-dimensional here, we can *also* recover the primal weight vector w explicitly — impossible with the Gaussian RBF kernel whose Φ lives in an infinite-dimensional space.

The support-vector expansion

The decision function is:

$$f(x) = \sum_{i \in \text{SV}} \alpha_i y_i k(x_i, x) + b$$

Only support vectors (those with $\alpha_i > 0$) contribute. By KKT complementary slackness:

- $\alpha_i = 0$ — point lies strictly outside the margin; **not** a support vector.
- $0 < \alpha_i < C$ — **free SV**, sits exactly on the margin, slack $\xi_i = 0$.
- $\alpha_i = C$ — **bounded SV**, inside the margin or misclassified, slack $\xi_i > 0$.

1. Imports

```
import cvxpy as cp
import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

cvxpy is a domain-specific language for convex optimisation. It lets us write the SVM dual as a literal mathematical expression and hand it to a convex QP solver (CLARABEL by default), bypassing the SMO algorithm libsvm uses internally.

numpy provides the array linear algebra: dot products, broadcasting, matrix multiplications. The Gram matrix $K = X X^T$ is a single @ operation.

pandas is used only to assemble the synthetic dataset with labelled columns — not strictly necessary for the math.

roc_auc_score is the rank-based area under the ROC curve, the right evaluation metric when classes are imbalanced (true positive rate vs false positive rate is independent of the class prior).

train_test_split is sklearn's stratified random splitter. **StandardScaler** standardises each feature column to mean 0 and standard deviation 1 — mandatory for distance-based kernels.

SVC is sklearn's reference implementation (calling libsvm under the hood); we train it with identical hyperparameters and verify the from-scratch solution matches.

2. Reproducibility

```
RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
```

A single integer seeds every random draw downstream: borrower features, default labels, train/test split. Fixing the seed makes the from-scratch $\leftrightarrow$ sklearn comparison deterministic — otherwise the QP would see two different training sets and we could not directly compare α , w , or b .

`np.random.default_rng()` returns a modern **Generator** object (PCG64 algorithm). Prefer this over the legacy `np.random.seed` + module-level calls.

3. Synthetic credit dataset

```
n_samples = 500

debt_to_income      = rng.beta(2, 5, n_samples) * 1.2
credit_utilization  = rng.beta(2, 4, n_samples)
missed_payments     = rng.poisson(0.6, n_samples)
savings_ratio       = rng.beta(2, 5, n_samples) * 0.5
income_volatility   = np.abs(rng.normal(0.15, 0.1, n_samples))
```

Five risk factors drawn from distributions chosen to roughly mimic a real retail-credit book. Beta distributions for bounded ratios; Poisson for a count variable; truncated normal for a non-negative continuous one.

Logistic data-generating process

```
logit = (  
    -4.0  
    + 8.0 * debt_to_income  
    + 5.0 * credit_utilization  
    + 2.0 * missed_payments  
    - 8.0 * savings_ratio  
    + 5.0 * income_volatility  
)  
p_default = 1.0 / (1.0 + np.exp(-logit))  
y_01 = (rng.uniform(size=n_samples) < p_default).astype(int)  
y_signed = 2 * y_01 - 1
```

The default label comes from a known logistic model. Each risk factor contributes additively to the **log-odds**:

$$\eta_i = \beta_0 + \beta_1 x_{i,1} + \cdots + \beta_5 x_{i,5}$$

which the sigmoid maps to a probability in $[0, 1]$:

$$p_i = \sigma(\eta_i) = \frac{1}{1 + e^{-\eta_i}}$$

Each borrower's label is then a Bernoulli draw $y_i \sim \text{Bernoulli}(p_i)$, implemented via the inverse-transform trick ($U < p$): if $U \sim \text{Uniform}(0, 1)$ then $\mathbf{P}(U < p) = p$, which means $1\{U < p\} \sim \text{Bernoulli}(p)$ exactly.

Coefficients were deliberately strengthened (compare $8.0 \cdot \text{DTI}$ vs the 3.0 used in `svm_minimal.py`) so the classes are clearly linearly separable and the linear SVM has a non-degenerate optimum to find.

The line `y_signed = 2 * y_01 - 1` converts labels from $\{0, 1\}$ to $\{-1, +1\}$ — the convention used throughout the slide math (where y_i multiplies the decision function $f(x)$).

Pitfall: If the logit coefficients are too weak, the soft-margin SVM degenerates to $w \approx 0$ (predict-all-majority) and the comparison to sklearn is dominated by solver noise rather than the true optimum.

4. Stratified train/test split

```
(X_train, X_test,
 y_train_01, y_test_01,
 y_train, y_test) = train_test_split(
    X_dataframe.values, y_01, y_signed,
    test_size=0.25, stratify=y_01, random_state=RANDOM_STATE,
)
```

`train_test_split` accepts any number of parallel arrays as positional arguments and splits them in lockstep with the same row indices. Three inputs (X , y in $\{0,1\}$, y in $\{-1,+1\}$) produce six aligned outputs.

Why keep both label conventions? Different parts of the script need different ones: `stratify=y_01` wants the binary form, the slide math uses ± 1 , and percentage reporting reads more naturally in $\{0, 1\}$.

Stratification preserves class balance

Without stratification, the split is uniform over rows and the default rate in each half drifts from the marginal. With `stratify=y_01`, sklearn samples within each class separately:

$$\Pr(y = 1 \mid \text{train}) = \Pr(y = 1 \mid \text{test}) = \Pr(y = 1)$$

verified in the run output by:

```
Default rate : train 65.60%    test 65.60%
```

Pitfall: Without stratification on small n , you can get unlucky splits like 70%/50% defaults — making the test set look either too hard or too easy and biasing every metric.

5. Feature scaling

```
scaler          = StandardScaler().fit(X_train)
X_train_scaled  = scaler.transform(X_train)
X_test_scaled   = scaler.transform(X_test)
```

Each column is remapped via the z-score transformation:

$$x'_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j}$$

where (μ_j, σ_j) are computed from the **training set only** — never from the test set or the full data. After this:

$$\bar{x}'_{:,j} = 0, \quad s'_{:,j} = 1$$

Why scaling is mandatory for SVMs

Every kernel measures distances or dot products between feature vectors. With the linear kernel:

$$k(x, x') = \langle x, x' \rangle = \sum_j x_j x'_j$$

If one column has range 1–40 (years) and another has range 0–1 (a ratio), the years column dominates every dot product and the other features are invisible to the kernel. Scaling puts every column on the same numeric footing.

Pitfall: Fit the scaler on **X_train only**, then transform both. Fitting on the full X before splitting leaks test-set statistics into training and makes evaluation optimistic.

6. Hyperparameter C

`C_value = 10.0`

C is the only hyperparameter the linear SVM has — the RBF kernel adds γ , polynomial adds (γ, r, d) , but linear has none of those. C controls the trade-off between flatness (small $\|w\|^2$) and slack-violation tolerance:

$$\frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$

- **C small** — flatness wins. Wide margin, many violations tolerated, simple boundary, high bias.
- **C large** — violation penalty wins. Narrow margin, few violations, complex boundary, high variance.

In the *dual*, C appears as the upper box bound $0 \leq \alpha_i \leq C$, i.e. each training point can exert at most C units of influence on the decision boundary.

Pitfall: C = 1 turned out to be too soft on this dataset — the SVM found $w \approx 0$ (predict-all-majority is cheaper than any non-zero w) and the comparison with sklearn became noise-dominated. C = 10 forces a meaningful non-degenerate optimum.

7. Kernel (Gram) matrix

```
K_train = X_train_scaled @ X_train_scaled.T
```

For the linear kernel, the Gram matrix is literally one matrix multiplication. Element (i, j) is the dot product between training rows i and j:

$$K_{ij} = \langle x_i, x_j \rangle = (XX^\top)_{ij}$$

Shape: (n_train, n_train) = (375, 375). The matrix is positive semi-definite by construction (every Gram matrix of real vectors is) — which is what makes the dual a **convex** QP.

This is the entire kernel trick

The dual depends on the data *only* through pairwise dot products. Once K is precomputed, the optimiser never touches X again — it operates purely on K and y. Swap this one line for

```
K_train = rbf_kernel(X_train_scaled, X_train_scaled, gamma=g)
```

and you have an RBF SVM with everything below unchanged. That is the kernel trick in action: changing the implicit feature space Φ without touching the dual machinery.

$$k(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$$

8. The Lagrangian dual as a convex QP

```
alpha = cp.Variable(n_train, nonneg=True)
alpha_y = cp.multiply(alpha, y_train)

dual_objective = cp.Maximize(
    cp.sum(alpha)
    - 0.5 * cp.quad_form(alpha_y, cp.psd_wrap(K_train))
)
dual_constraints = [
    alpha <= C_value,
    cp.sum(alpha_y) == 0,
]

dual_problem = cp.Problem(dual_objective, dual_constraints)
dual_problem.solve()
```

This is the heart of the script. The Part-II slide formula

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij} \\ \text{s. t.} \quad & \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C \end{aligned}$$

is encoded literally in cvxpy.

Line-by-line

`alpha = cp.Variable(n_train, nonneg=True)` — one decision variable per training row, with the lower-box constraint $\alpha_i \geq 0$ baked in.

`alpha_y = cp.multiply(alpha, y_train)` — the element-wise product $(\alpha_i y_i)$ appears everywhere: inside the quadratic form, inside the equality constraint, and (after solving) as `sklearn's dual_coef_`.

`cp.sum(alpha)` = $\sum_i \alpha_i$, the linear term. Without regularisation the optimum would push every α_i to the upper bound `C`.

`cp.quad_form(alpha_y, K_train)` = $(\alpha^\circ y)^\top K (\alpha^\circ y)$ is the quadratic regulariser. Substituting the stationarity condition $w = \sum_i \alpha_i y_i x_i$:

$$\|w\|^2 = (\alpha \circ y)^\top K (\alpha \circ y) = \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij}$$

So this term is $\|w\|^2$ written in dual form. Maximising $\sum \alpha_i - \frac{1}{2} \|w\|^2$ implements the slide's soft-margin trade-off.

`cp.psd_wrap(K_train)` — asserts to `cvxpy` that `K` is PSD. Mathematically true (it's a Gram matrix), but floating-point can leave a few eigenvalues at -10^{-16} instead of exactly zero, which `cvxpy` would reject. The `wrap` says “trust me”.

`alpha <= C_value` — the upper box. Together with the lower bound from `nonneg=True` this gives $0 \leq \alpha_i \leq C$, the signature of the soft-margin formulation. A hard-margin SVM would have no upper bound.

`cp.sum(alpha_y) == 0` — the equality constraint from $\partial L / \partial b = 0$. Forces a balanced α allocation between the two classes.

`dual_problem.solve()` — `cvxpy` canonicalises the problem into standard QP form and hands it to a solver (CLARABEL by default for QPs). After return:

- `dual_problem.status` → “*optimal*” if it converged.
- `dual_problem.value` → the optimal objective, equal to the primal optimum by strong duality.
- `alpha.value` → the optimal α vector as NumPy.

9. Support-vector identification via KKT

```
TOL = 1e-5
sv_mask = alpha_value > TOL
sv_indices = np.where(sv_mask)[0]

free_mask = (alpha_value > TOL) & (alpha_value < C_value - TOL)
free_indices = np.where(free_mask)[0]
bounded_indices = np.where(alpha_value >= C_value - TOL)[0]
```

KKT **complementary slackness** partitions training points by their dual variable:

$$\alpha_i = 0 \Rightarrow \text{not an SV}$$

$$0 < \alpha_i < C \Rightarrow \text{free SV } (\xi_i = 0)$$

$$\alpha_i = C \Rightarrow \text{bounded SV } (\xi_i > 0)$$

Numerical tolerance $TOL = 1e-5$ is the threshold for “effectively zero” vs “effectively C” in floating point. The QP solver returns α values that satisfy the KKT conditions up to its convergence tolerance, not exactly.

The slide claim — “the model complexity is set by the number of support vectors, not by the dimension of x ” — is verified by this partition: the decision function depends on at most $|SV|$ terms, not on n_{train} .

10. Recovering the primal weight vector w

```
alpha_y_train = alpha_value * y_train
w_scratch = alpha_y_train @ X_train_scaled

b_per_free_sv = y_train[free_indices] - X_train_scaled[free_indices] @ w_scratch
b_scratch = float(np.mean(b_per_free_sv))
```

From the stationarity condition $\partial L / \partial w = 0$:

$$w = \sum_{i \in SV} \alpha_i y_i x_i$$

In NumPy: $(\alpha \cdot y) @ X$, with broadcasting handling the $\alpha_i y_i x_i$ product. The non-SVs contribute zero so summing over all training rows gives the same answer as summing over SVs only.

Why this is impossible for RBF kernels

With the linear kernel $\Phi(x) = x$, so w lives in the same **finite-dimensional** input space as x and has a concrete representation. For the Gaussian RBF kernel, Φ lives in an **infinite-dimensional** reproducing-kernel Hilbert space — w would have infinitely many entries and cannot be materialised. The decision function still works, but only via the dual form $\sum_i \alpha_i y_i k(x_i, x)$.

Recovering the bias b

For every free SV i (those with $0 < \alpha_i < C$), the margin constraint is exactly tight: $y_i(\langle w, x_i \rangle + b) = 1$. Solving for b :

$$b = y_i - \langle w, x_i \rangle \quad (\text{any free SV})$$

In principle, any single free SV gives b . Averaging across all free SVs cancels solver-tolerance noise:

$$b = \frac{1}{|\text{free SVs}|} \sum_{i \in \text{free SVs}} (y_i - \langle w, x_i \rangle)$$

Pitfall: If the QP solver returns no free SVs (only bounded ones, e.g. in pathological soft-margin regimes), b cannot be recovered by this formula. libsvm handles this with a more elaborate KKT balancing scheme.

11. Decision function — primal and dual forms

```
# Primal form (only possible with linear kernel):
f_primal = X_test_scaled @ w_scratch + b_scratch

# Dual form (support-vector expansion):
K_test_sv = X_test_scaled @ X_train_scaled[sv_indices].T
f_dual    = K_test_sv @ alpha_y_train[sv_indices] + b_scratch
```

Two algebraically identical expressions:

$$f(x) = \langle w, x \rangle + b \quad (\text{primal})$$

$$f(x) = \sum_{i \in \text{SV}} \alpha_i y_i \langle x_i, x \rangle + b \quad (\text{dual})$$

Substituting $w = \sum_i \alpha_i y_i x_i$ into the primal recovers the dual:

$$\langle w, x \rangle = \langle \sum_i \alpha_i y_i x_i, x \rangle = \sum_i \alpha_i y_i \langle x_i, x \rangle$$

Verified numerically by:

```
max |f_primal - f_dual| = 4.88e-13    (machine epsilon)
```

The dual form generalises to any kernel by replacing the inner product with $k(x_i, x)$; the primal form does not, because w may not exist in input space.

12. sklearn baseline

```
svc = SVC(kernel="linear", C=C_value)
svc.fit(X_train_scaled, y_train)

w_sklearn = svc.coef_.ravel()
b_sklearn = svc.intercept_[0]
```

Train sklearn's reference SVC with the *same* kernel and the *same* C. Under the hood sklearn calls libsvm, which uses **Sequential Minimal Optimisation** (SMO, Platt 1998) — iteratively picks pairs (α_i, α_j) and solves a 2-variable analytical subproblem until convergence.

sklearn attributes mapped to slide notation

- `svc.coef_` $\leftrightarrow w$ (only exists for the linear kernel).
- `svc.intercept_[0]` $\leftrightarrow b$.
- `svc.dual_coef_[0]` $\leftrightarrow \alpha_i y_i$ (signed dual coefficients).
- `svc.support_` $\leftrightarrow$ indices of SVs.
- `svc.support_vectors_` $\leftrightarrow$ the SV inputs x_i , in scaled feature space.

13. Test-set comparison

```
f_sklearn = svc.decision_function(X_test_scaled)

max_err = np.abs(f_dual - f_sklearn).max()
mean_err = np.abs(f_dual - f_sklearn).mean()
w_err = np.abs(w_scratch - w_sklearn).max()
b_err = abs(b_scratch - b_sklearn)
```

Both solvers attack the same convex QP, so by strong duality and uniqueness of w (when K is sufficiently full-rank) the results must agree. Small numerical disagreements remain because CLARABEL (interior-point) and SMO (working-set descent) traverse different paths to the optimum and stop at different convergence tolerances.

Empirically the from-scratch run reports:

```
max |w_scratch - w_sklearn|      = 1.86e-03
|b_scratch - b_sklearn|         = 6.90e-04
max |f_scratch - f_sklearn|      = 8.47e-03
mean |f_scratch - f_sklearn|     = 2.41e-03

Test accuracy : from-scratch 0.840   sklearn 0.840
Test ROC-AUC  : from-scratch 0.922   sklearn 0.922
```

Predictions and AUC are bit-identical. Margin scores agree to ~1% — pure solver-tolerance noise.

14. Scoring a brand-new borrower

```
new_borrower = np.array([[0.55, 0.78, 2, 0.05, 0.30]])
new_scaled = scaler.transform(new_borrower)
new_margin = float((new_scaled @ w_scratch)[0] + b_scratch)
new_decision = "DEFAULT" if new_margin >= 0 else "REPAY"
```

Inference at deployment time uses the **primal form** — once w is known, scoring a new row x is just one dot product $\langle w, x \rangle + b$. That is $O(d)$ where d is the number of features, independent of the training set size. The dual form would require summing kernel evaluations over every SV, $O(|SV| \cdot d)$ — far slower for high-traffic prediction.

Critical detail: `scaler.transform(new_borrower)` applies the *same* μ_j, σ_j learned from X_{train} . The new borrower must enter the model in the same feature space the model was trained in.

Pitfall: In production this scaler must be persisted alongside the model. Use `joblib.dump` on the full `Pipeline(scaler, classifier)` — never on the classifier alone.

15. Why this exercise matters

Calling `SVC().fit()` hides every interesting object in the SVM behind a single function call. By writing the dual QP explicitly we have made the Lagrangian **visible**:

- The *variables* are α_i — not w or b directly. b is recovered from the KKT margin equation; w from stationarity.
- The *objective* is $\sum \alpha_i - \frac{1}{2} \|w\|^2$, which trades off the number of “active” training points against the magnitude of w .
- The *constraints* — $0 \leq \alpha_i \leq C$ and $\sum \alpha_i y_i = 0$ — encode the soft-margin slack budget and the bias degree of freedom respectively.
- The *solution* obeys KKT complementary slackness, which is what creates the sparse support-vector expansion. The slide's claim about complexity scaling with $|SV|$ follows immediately.

Once these objects are explicit, generalising is trivial: replace the linear-kernel Gram with an RBF Gram and the same code solves the Gaussian-kernel SVM. Add a second set of multipliers α_i^* on the lower tube constraint and you have SVR (Support Vector Regression). Both extensions are a small edit to the same dual QP.

End of document.